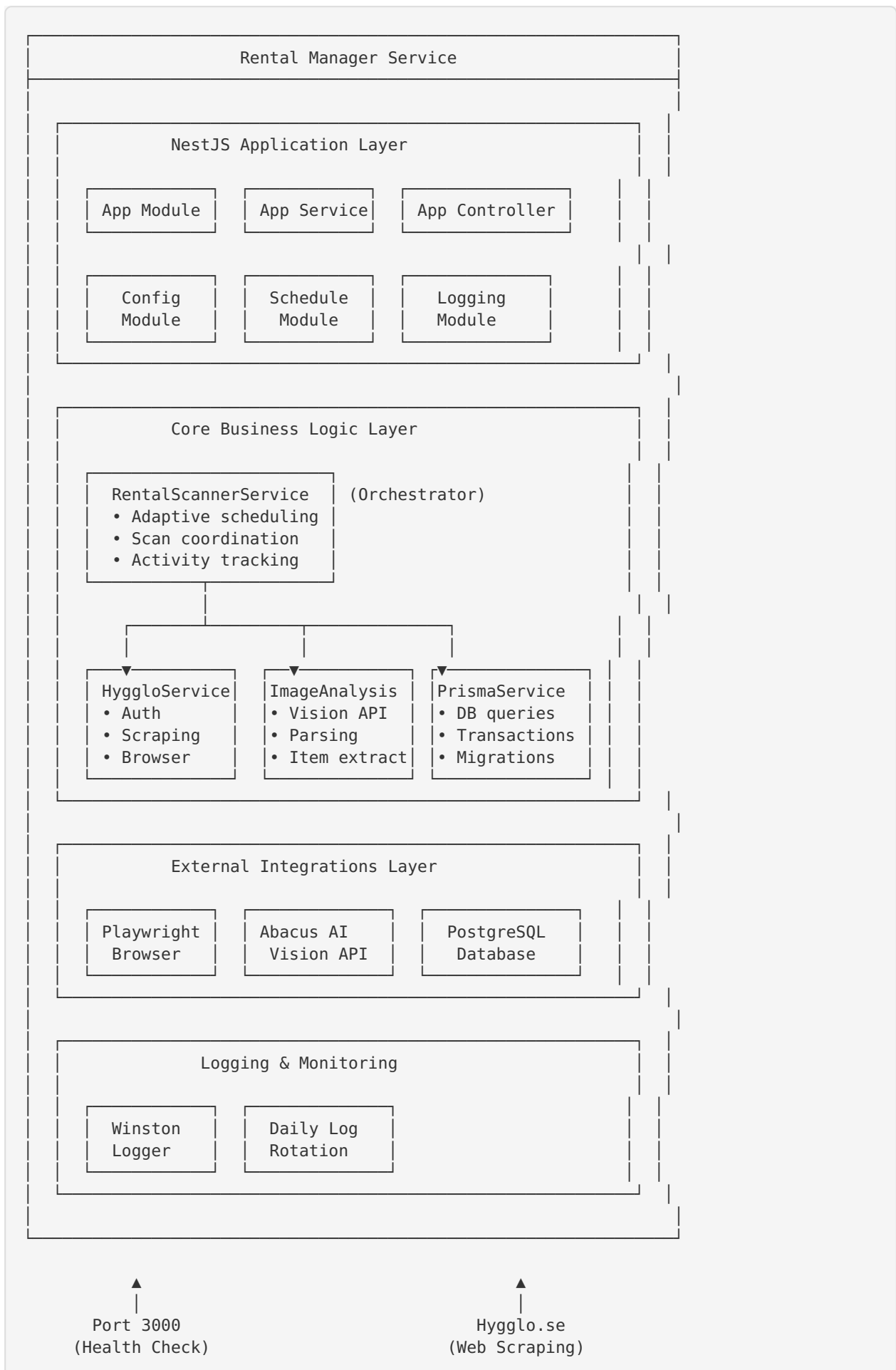


Rental Manager - Architecture Documentation

System Overview

The Rental Manager is a headless background service that automates rental tracking on the Hygglo platform. It uses browser automation to scrape rental data, AI/ML for image analysis, and adaptive scheduling to balance real-time tracking with resource efficiency.

Architecture Diagram



Component Details

1. RentalScannerService (Core Orchestrator)

Responsibilities:

- Coordinates the entire scanning workflow
- Implements adaptive scheduling logic
- Manages activity tracking and interval switching
- Orchestrates data processing pipeline

Adaptive Scheduling Algorithm:

1. Start: `scanInterval` = 1 minute
2. On each scan:
 - a. Scan rentals (ongoing + upcoming)
 - b. Count `new` rentals found
 - c. If `newRentals` > 0:
 - Reset `lastActivityTime` = `now()`
 - `Set` `scanInterval` = 1 minute
 - d. If `newRentals` == 0:
 - Calculate `timeSinceLastActivity`
 - If `timeSinceLastActivity` > 30 minutes:
 - `Set` `scanInterval` = 5 minutes
3. Schedule next scan after `scanInterval`
4. Repeat `from` step 2

Key Methods:

- `onModuleInit()` : Bootstrap scanner on app start
- `performScan()` : Execute complete scan cycle
- `processRental(rental)` : Process individual rental
- `scheduleNextScan()` : Schedule next scan iteration
- `getStatus()` : Return current scanner status

2. HyggloService (Browser Automation)

Responsibilities:

- Authenticate with Hygglo using Playwright
- Maintain persistent browser session
- Scrape rental data from ongoing/upcoming pages
- Fetch detailed listing information
- Handle re-authentication on session expiry

Browser Automation Flow:

1. Launch Chromium (headless)
2. Navigate `to` login page
3. Fill credentials
4. Submit `form`
5. `Verify` authentication
6. Navigate `to` rental pages
7. Extract `listing` cards
8. Parse `data` from DOM
9. Clean up session

Key Methods:

- `authenticate()` : Login to Hygglo
- `scanRentals(status)` : Scan ongoing or upcoming rentals
- `fetchListingDetails(url)` : Get full listing info
- `ensureAuthenticated()` : Check/refresh session
- `cleanup()` : Close browser resources

Selectors Used:

- `[data-testid="rental-card"]` : Rental cards
- `.rental-item` : Alternative rental card selector
- `a[href*="/listing/"]` : Listing URLs
- `.description` : Listing description
- `img` : Photos

3. ImageAnalysisService (AI Vision)

Responsibilities:

- Analyze rental photos using Vision AI
- Extract items with confidence scores
- Parse descriptions for item keywords
- Deduplicate items across multiple photos

Image Analysis Flow:

1. Receive image URL(s)
2. Call Abacus AI Vision API
3. Request JSON response with items + confidence
4. Parse **and** validate response
5. Deduplicate across photos (keep max confidence)
6. **Return** extracted items

AI Prompt:

Analyze this image and list all visible items, objects, furniture, equipment, or belongings. **For** each item, provide a confidence score (0-1) indicating how certain you are about its presence. Please respond in JSON format with an array of objects containing **"itemName"** and **"confidenceScore"** fields.

Key Methods:

- `analyzeImage(url)` : Analyze single image
- `analyzeImages(urls)` : Batch analyze with deduplication
- `parseDescriptionForItems(text)` : Extract items from text

Rate Limiting:

- 1 second delay between image API calls
- Prevents API throttling

4. PrismaService (Database Layer)

Responsibilities:

- Manage database connections
- Execute queries and transactions

- Handle connection lifecycle
- Provide type-safe database access

Key Features:

- Auto-connect on module init
- Auto-disconnect on shutdown
- Connection pooling
- Query logging (debug mode)

5. LoggingService (Observability)

Responsibilities:

- Structured logging with Winston
- Daily log rotation
- Multi-transport (console + file)
- Contextual metadata

Log Transports:

- **Console:** Colored, formatted for development
- **File:** JSON format, daily rotation, 14-day retention

Log Structure:

```
{
  "timestamp": "2026-01-29 22:40:23",
  "level": "info",
  "message": "Scan completed",
  "duration": 1234,
  "newRentals": 2
}
```

Data Flow

Complete Scan Cycle

1. Timer triggers scan
↓
2. RentalScannerService.performScan()
↓
3. HyggloService.scanRentals('ongoing')
 - ↳ Browser automation
 - ↳ DOM parsing
 - ↳ Return rental data
 ↓
4. HyggloService.scanRentals('upcoming')
 - ↳ Browser automation
 - ↳ DOM parsing
 - ↳ Return rental data
 ↓
5. For each rental:
 - ↳ Check if exists in DB (by listing_id)
 - ↳ If new:
 - ↳ Fetch full details
 - ↳ Save to database
 - ↳ ImageAnalysisService.analyzeImages()
 - ↳ Call Vision API for each photo
 - ↳ Parse JSON response
 - ↳ Save to extracteditem table
 - ↳ parseDescriptionForItems()
 - ↳ Keyword matching
 - ↳ Save to itemcatalog table
 - ↳ If exists: Update status/info
 ↓
6. Count new rentals
↓
7. Update activity tracking
↓
8. Adjust scan interval if needed
↓
9. Schedule next scan

Database Schema

rental table:

```
CREATE TABLE rental (
  id UUID PRIMARY KEY,
  listing_id VARCHAR UNIQUE NOT NULL,
  title VARCHAR NOT NULL,
  status VARCHAR NOT NULL,
  start_date TIMESTAMP,
  end_date TIMESTAMP,
  renter_info VARCHAR,
  listing_url VARCHAR NOT NULL,
  description TEXT,
  photos_urls VARCHAR[],
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);
```

extracteditem table:

```
CREATE TABLE extracteditem (
  id UUID PRIMARY KEY,
  rental_id UUID REFERENCES rental(id) ON DELETE CASCADE,
  item_name VARCHAR NOT NULL,
  source VARCHAR NOT NULL, -- 'photo' or 'description'
  confidence_score FLOAT,
  created_at TIMESTAMP DEFAULT NOW()
);
```

itemcatalog table:

```
CREATE TABLE itemcatalog (
  id UUID PRIMARY KEY,
  listing_id VARCHAR REFERENCES rental(listing_id) ON DELETE CASCADE,
  item_name VARCHAR NOT NULL,
  description TEXT,
  first_seen_at TIMESTAMP DEFAULT NOW(),
  UNIQUE(listing_id, item_name)
);
```

Indexes:

- rental.listing_id (unique)
- rental.status
- extracteditem.rental_id
- extracteditem.source
- itemcatalog.listing_id

Deployment Architecture

Docker Container Structure

```
rentals-manager/
├── app/
│   ├── dist/           # Compiled TypeScript
│   ├── node_modules/   # Production dependencies
│   ├── prisma/         # Prisma schema & client
│   └── logs/           # Rotating log files (mounted volume)
├── /usr/bin/chromium   # System browser for Playwright
└── NODE_ENV=production
```

Container Lifecycle

1. Build Phase:

- Install Node.js dependencies
- Install Chromium and system deps
- Generate Prisma Client
- Compile TypeScript to JavaScript

2. Runtime Phase:

- Load environment variables
- Connect to PostgreSQL
- Initialize logging

- Start NestJS application
- Bootstrap RentalScannerService
- Begin adaptive scanning loop

3. Shutdown Phase:

- Receive SIGTERM signal
- Close browser sessions
- Disconnect database
- Flush log buffers
- Exit cleanly

AWS Deployment Options

Option 1: EC2 + Docker Compose

- Single EC2 instance (t3.medium)
- Docker Compose with PostgreSQL
- Persistent EBS volume for logs/database
- Elastic IP for static address
- Security group: 3000 (health check)

Option 2: ECS Fargate

- Serverless container execution
- RDS PostgreSQL (managed)
- CloudWatch for logs
- EFS for persistent storage
- Application Load Balancer (optional)

Option 3: ECS EC2

- ECS cluster on EC2 instances
- RDS PostgreSQL (managed)
- CloudWatch for logs
- EBS volumes for storage
- Auto Scaling Group

Performance Characteristics

Resource Usage

- **Memory:** 200-400 MB typical, 800 MB peak during image analysis
- **CPU:** Low (5-10%) during idle, moderate (30-50%) during scans
- **Network:** Minimal, spikes during image downloads
- **Disk:** ~100 MB application, logs grow ~10 MB/day
- **Database:** Minimal queries, mostly inserts

Scaling Considerations

Vertical Scaling:

- Increase container memory for more concurrent image analysis
- Increase CPU for faster browser automation

Horizontal Scaling:

- NOT RECOMMENDED: Browser automation has state
- If needed: Implement distributed locking

Database Scaling:

- Current load: Very light
- Indexes handle queries efficiently
- Consider read replicas if adding analytics

Bottlenecks

1. **Image Analysis API:** Rate limited, sequential processing
2. **Browser Automation:** Single session, serial page visits
3. **Network:** Dependent on Hygglo response times

Optimization Opportunities

1. **Parallel Image Analysis:** Process multiple images concurrently (with rate limiting)
2. **Caching:** Cache listing details for unchanged rentals
3. **Incremental Scans:** Only fetch new/updated rentals
4. **Database Pooling:** Already implemented via Prisma

Security Considerations

Credentials Management

- **Environment Variables:** All secrets in .env
- **No Hardcoding:** Credentials never in code
- **Docker Secrets:** Use for production deployments
- **AWS Secrets Manager:** Recommended for cloud deployments

Network Security

- **Database Access:** VPC-only for production
- **HTTPS:** Required for API communication
- **Firewall Rules:** Minimal open ports
- **No Public Database:** PostgreSQL not exposed

Application Security

- **No User Input:** Headless service, no user-facing API
- **Sanitization:** All scraped data sanitized before DB insert
- **Error Handling:** No sensitive data in error messages
- **Logging:** Credentials masked in logs

Monitoring & Observability

Health Checks

- **Endpoint:** GET /health
- **Response:** JSON with status, uptime, scanner state
- **Use Cases:** Docker healthcheck, load balancer, monitoring

Key Metrics to Monitor

1. **Scan Success Rate:** % of successful scans
2. **New Rentals Per Hour:** Activity level
3. **Authentication Failures:** Session issues

4. **Image Analysis Errors:** API issues
5. **Database Query Time:** Performance
6. **Memory Usage:** Resource leaks

Log Analysis

Important Log Messages:

- Starting rental scanner service : Bootstrap
- New activity detected : Rentals found
- Switching to reduced scanning : Inactivity threshold hit
- Authentication failed : Credentials/session issue
- Image analysis failed : API error
- Database connection failed : DB issue

Alerting Recommendations

1. **Critical:** Authentication failures (3+ consecutive)
2. **Warning:** No new rentals for 24 hours
3. **Info:** Scan interval changes
4. **Error:** Database connection failures

Testing Strategy

Unit Tests

- Service logic (mocked dependencies)
- Description parsing
- Item deduplication
- Scheduling algorithm

Integration Tests

- Database operations
- API communication
- End-to-end scan cycle (mocked browser)

E2E Tests

- Full scan with real browser
- Authentication flow
- Data persistence

Manual Testing

- Local development with test credentials
- Docker build and run
- Health endpoint verification
- Log inspection

Troubleshooting Guide

Common Issues

Issue: Browser launch fails

- Check Chromium installed in container
- Verify system dependencies
- Check memory availability

Issue: Authentication fails

- Verify credentials correct
- Check Hygglo site structure changes
- Review login page selectors

Issue: No items extracted

- Verify API key set
- Check API quota
- Review API response format

Issue: High memory usage

- Check browser sessions closed
- Verify log rotation working
- Monitor Prisma connection pool

Future Enhancements

Short Term

- [] Retry logic for failed scans
- [] Prometheus metrics export
- [] GraphQL API for querying data
- [] Email notifications

Medium Term

- [] Multi-account support
- [] Enhanced item categorization
- [] Webhook support
- [] Real-time dashboard

Long Term

- [] Machine learning item classification
- [] Predictive analytics
- [] Mobile app integration
- [] Multi-platform support (beyond Hygglo)

Document Version: 1.0

Last Updated: 2026-01-29

Maintainer: Bananajoexxc